

# The self-critique loop

Autonomous Social Media Content Studio — backend architecture reference

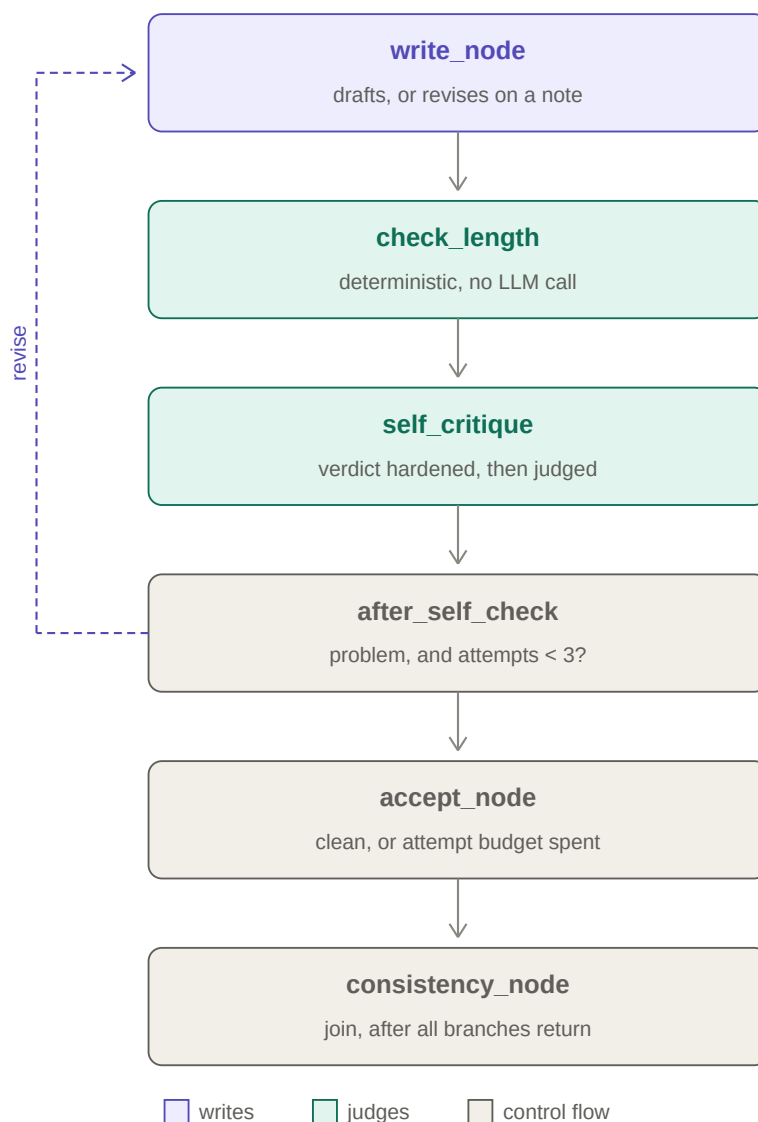
Every box and every arrow in the per-platform generation loop, mapped to the code that implements it.

## What this document covers

The backend generates one post per platform. Each platform runs its own independent write → check → revise loop, and all three loops run at the same time inside a single LangGraph superstep. This document walks the loop for **one** platform branch.

The loop implements the **Reflection** pattern (also called Self-Refine): the same model both writes the draft and judges it, under two different prompts. There is no separate critic model and no human in the loop — the quality gate is the writer reviewing its own output, bounded by a fixed attempt budget so it always terminates.

## The loop at a glance



Purple boxes produce text. Teal boxes judge text. Grey boxes route control flow and do not call a model. The dashed return edge is the revision path.

## Part 1 — What each box does

Six boxes, in execution order. Each entry names the file it lives in, what it reads, what it writes back into graph state, and the failure it was designed around.

### 1. write\_node

The only box that produces a draft. It handles both the first attempt and every revision, distinguished purely by whether `state["problem"]` is set.

|                 |                                                                                                                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| File            | src/node/writer_node.py                                                                                                                                                                                           |
| Reads           | platform, brief, attempts, problem, draft                                                                                                                                                                         |
| Writes          | draft, attempts, action_log, trace_events                                                                                                                                                                         |
| LLM call        | Yes — one, via <code>agent.write_post()</code> . Tier is chosen from the brief, so an easy topic routes to the cheap local model and a hard one to Azure.                                                         |
| Two modes       | <b>Fresh:</b> problem is empty → write from the brief alone.<br><b>Revise:</b> problem is set → the previous draft <i>and</i> the critic's note are both passed in, so the model edits rather than starting over. |
| Attempt counter | attempts is incremented here, not in the checker — a draft that was never written cannot have been attempted.                                                                                                     |

Why it re-reads the old draft on a revision: a model asked to "make it shorter" with no sight of what it wrote produces a different post, not a shorter one. Passing `old_draft` turns generation into editing, which is what preserves the parts the critic did not complain about.

### 2. check\_length

The first gate, and the cheap one. Pure Python — no model, no network, no cost.

|               |                                                                                                                                                                         |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| File          | src/agent.py (called from src/node/self_check_node.py)                                                                                                                  |
| Reads         | platform, draft                                                                                                                                                         |
| Returns       | an empty string when the draft fits, or a specific instruction when it does not                                                                                         |
| Limits        | LinkedIn 3000 · X 280 · Instagram 2200 characters, read from the RULES table                                                                                            |
| Short-circuit | If this gate fails, <code>self_critique</code> is <b>skipped entirely</b> for that attempt. There is no point paying for an opinion on a draft that is already invalid. |

The message it returns is deliberately actionable rather than descriptive — it becomes the revision instruction handed straight back to `write_node`:

```
Too long: 412 characters, limit is 280.  
Cut at least 132 characters. Keep the hook.
```

"Keep the hook" is there because the obvious way for a model to satisfy a character limit is to truncate the ending, which destroys the opening line the post depends on.

### 3. self\_critique

The reflection step. The writer is re-prompted as a reviewer and asked for a verdict on its own draft: either the single word OK, or one short imperative sentence naming the problem.

|         |                                                                    |
|---------|--------------------------------------------------------------------|
| File    | src/agent.py                                                       |
| Reads   | platform, brief, draft, attempt, previous_problem                  |
| Returns | an empty string to accept, or a one-sentence instruction to revise |

|                     |                                                                                                               |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Model tier</b>   | Pinned to the strong tier. "critique" is in PINNED_TASKS in configs/llms.py — for capability, not difficulty. |
| <b>Sees history</b> | On attempt 2 and later the critic is shown <b>its own previous note</b> .                                     |
| <b>Final pass</b>   | On the last attempt the prompt adds a warning to accept unless the draft is factually wrong or off-brief.     |

**Why the critic sees its own last note.** Without it, every pass is a stateless review. The model raises the same complaint about the same draft indefinitely, the loop can only ever end by exhausting its attempts, and the post ships flagged with a note that was never actually unresolvable. Feeding the previous note forward turns a repeated review into a conversation about one specific problem.

**Why this call is pinned to the strong tier.** The critic must answer with exactly OK or exactly one instruction. Small local models tend to restate the checklist instead. That output never parses as OK, so every draft loops to the attempt cap — and worse, the restated text is what gets handed to the writer as its revision instruction. The critic gates the entire self-correction loop, so it is the wrong place to save a fraction of a cent.

### 3a. Verdict hardening — `_is_ok` and `_is_malformed`

Two guards sit between the raw model output and the rest of the loop. They exist because a verdict is free-text and the loop treats it as a control signal.

| Guard                      | Catches                                                                                                                  | Result                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Empty verdict              | The model returned nothing at all.                                                                                       | Accept. Logged at WARNING — silence is not evidence of a problem, but it should be visible.             |
| <code>_is_ok</code>        | Decorated approvals: OK., **OK**, or reasoning that ends with OK on its last line.                                       | Accept. Without stripping the decoration, a passing draft is sent back for a revision it does not need. |
| <code>_is_malformed</code> | A verdict over 400 characters, or one containing checklist markers such as ## Check — the critic echoing its own prompt. | Accept, and log at WARNING. This text must never reach <code>write_node</code> .                        |

All three guards fail **towards acceptance**, and that direction is deliberate. An unusable verdict is not evidence that the draft is bad — the draft has already passed the deterministic length gate. Treating garbage as a real critique is actively harmful, because the garbage becomes the instruction the writer is told to act on.

## 4. after\_self\_check

The decision point. Two inputs, two outcomes, no model call.

|                     |                                                                                                                        |
|---------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>File</b>         | src/routes/routing.py                                                                                                  |
| <b>Reads</b>        | problem, attempts                                                                                                      |
| <b>Rule</b>         | problem is set <b>and</b> attempts < 3 → revise. Otherwise → accept.                                                   |
| <b>MAX_ATTEMPTS</b> | 3. This is what guarantees the loop terminates: a model that is never satisfied still costs a bounded number of calls. |

Note that *accept* is reached by two very different routes: a clean draft the critic approved, and a draft that simply ran out of attempts with its problem still open. Both leave here on the same edge, and it is `accept_node` that tells them apart.

## 5. accept\_node

Publishes the finished draft out of the branch. It is the only box whose output escapes into the shared parent state.

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>File</b>          | src/node/accept_node.py                                                                                                                                                              |
| <b>Reads</b>         | platform, draft, attempts, problem                                                                                                                                                   |
| <b>Writes</b>        | drafts as a single-key dict, plus action_log and trace_events                                                                                                                        |
| <b>Clean exit</b>    | problem empty → status "accepted". Logged at INFO.                                                                                                                                   |
| <b>Degraded exit</b> | problem still set → status "accepted_as_is", and the unresolved note is recorded in the trace. Logged at <b>WARNING</b> , because this is the outcome a human might want to look at. |

It writes {platform: draft} — one key, its own. The merge\_drafts reducer on the parent state combines the three branches' single-key dicts into one complete set. That is what lets three branches finish at the same instant without a write conflict.

## 6. consistency\_node

The only check in the entire system that is **not** self-critique, and the reason the loop is not sufficient on its own.

|                    |                                                                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>File</b>        | src/node/consistency_node.py                                                                                                                           |
| <b>Reads</b>       | brief, drafts (all platforms)                                                                                                                          |
| <b>Writes</b>      | corrected drafts, action_log, trace_events                                                                                                             |
| <b>Runs</b>        | Once, after every branch has returned. LangGraph will not schedule it earlier.                                                                         |
| <b>On conflict</b> | Rewrites only the contradicting posts, via the same write_post() the loop uses. A failure on one post keeps that original and lets the others through. |

A writer cannot notice that its post contradicts a post it never saw. If the LinkedIn draft says "40% faster" and the X draft says "4x faster", both are individually defensible against the brief and both pass their own self-critique. Only a pass that reads all three together can catch it — which also makes this the natural join point for the parallel branches.

## Part 2 — What each arrow means

Seven edges. Two of them are decisions; the rest are unconditional.

| Edge                                                      | Condition                               | What travels along it                                                                                                                                                                     |
|-----------------------------------------------------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| fan_out → write_node                                      | Once per platform, all at the same time | A fresh PlatformState: the shared brief, this platform's id, and empty draft / problem / attempts. Emitted as a LangGraph Send, which is what starts the three branches in one superstep. |
| write_node → check_length                                 | Always                                  | The new draft and the incremented attempt number. Both gates live inside self_check_node, so this is a function call, not a graph edge.                                                   |
| check_length → self_critique                              | Only if the length gate passed          | Nothing new — the same draft, now known to be within the platform limit. If the gate failed, this edge is skipped and no model is called.                                                 |
| self_critique → after_self_check                          | Always                                  | The verdict, already hardened to either an empty string or one instruction sentence. This is written into state["problem"], overwriting the previous attempt's note.                      |
| after_self_check → write_node<br>(the dashed return edge) | problem set and attempts < 3            | The problem, the current draft, and the attempt count. write_node reads all three and switches into revise mode. This is the loop.                                                        |
| after_self_check → accept_node                            | No problem, or attempts exhausted       | The final draft plus whatever problem remains. A non-empty problem here is what marks the draft as accepted-as-is rather than clean.                                                      |
| accept_node → consistency_node                            | After all three branches return         | Only PlatformOutput: drafts, action_log, trace_events. Everything else — brief, draft, problem, attempts — stays behind in the branch.                                                    |

### Why the return edge is the interesting one

Every other edge moves forward. The dashed edge is what makes this a self-correcting system rather than a pipeline: it feeds the critic's judgement back into the generator as an instruction, and it is bounded so that a model which is never satisfied cannot run forever. Remove that edge and the backend becomes a single-shot generator with a length check.

## What crosses the branch boundary, and what does not

The loop runs inside a compiled subgraph with its own state type. That boundary is what makes running three platforms in parallel safe.

| Field                    | Scope                        | Why                                                                                                                                                                               |
|--------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| brief                    | Branch-local (copied in)     | Read-only for the whole branch. If it were returned, all three branches would write the same key in one step and LangGraph would raise <code>InvalidUpdateError</code> .          |
| draft, problem, attempts | Branch-local, never returned | The three fields the loop mutates on every pass. Keeping them inside <code>PlatformState</code> is exactly what stops LinkedIn's revision from overwriting X's draft mid-flight.  |
| drafts                   | Shared, merged               | Each branch contributes one key. The <code>merge_drafts</code> reducer combines them instead of letting the last writer win.                                                      |
| action_log               | Shared, appended             | Append-only via <code>operator.add</code> . Load-bearing: the follow-up router greps it for the string "refined on request:" to resolve messages like "do the same for LinkedIn". |
| trace_events             | Shared, appended             | The structured mirror of <code>action_log</code> — step, actor, attempt, status, duration. The UI sorts by timestamp to rebuild the real interleaved execution order.             |

## Part 3 — A worked run

One platform (X, 280 characters), three attempts, ending in a clean accept. Read top to bottom; each row is one traversal of a box.

| Box              | Attempt | What happened                                                                                     |
|------------------|---------|---------------------------------------------------------------------------------------------------|
| write_node       | 1       | problem is empty, so a fresh draft is written from the brief. Result: 412 characters.             |
| check_length     | 1       | $412 > 280$ . Gate fails. Returns "Too long: 412 characters... cut at least 132".                 |
| self_critique    | 1       | <b>Skipped.</b> No model call — the length gate already found a problem.                          |
| after_self_check | 1       | problem set, attempts $1 < 3 \rightarrow$ revise.                                                 |
| write_node       | 2       | Revise mode. The old draft and the length instruction are both passed in. Result: 265 characters. |
| check_length     | 2       | $265 \leq 280$ . Gate passes.                                                                     |
| self_critique    | 2       | Critic reviews and returns "The opening buries the number — lead with 40%."                       |
| after_self_check | 2       | problem set, attempts $2 < 3 \rightarrow$ revise.                                                 |
| write_node       | 3       | Revise mode, acting on the critic's note. Result: 251 characters.                                 |
| check_length     | 3       | Passes.                                                                                           |
| self_critique    | 3       | Final pass, and the critic is shown its own previous note. Returns OK.                            |
| after_self_check | 3       | problem empty $\rightarrow$ accept.                                                               |
| accept_node      | 3       | Publishes {"x": draft}. Status "accepted", logged at INFO.                                        |
| consistency_node | —       | Runs once all three platforms have arrived. Compares the posts against each other.                |

Total cost for this branch: three writer calls and two critic calls. The first attempt's critic call was saved by the length gate — that short-circuit is worth roughly one model call per over-long draft, on every platform, on every run.

### If attempt 3 had failed instead

after\_self\_check would still route to accept, because the attempt budget is spent. accept\_node would then take its second path: status "accepted\_as\_is", the unresolved note recorded in the trace and shown in the UI, and a WARNING in the log naming the platform and the note. The post still ships. Shipping a flagged post is a smaller loss than failing the whole request.

# Where to look when something goes wrong

| Symptom                        | Most likely cause                                                                           | Log line to search for        |
|--------------------------------|---------------------------------------------------------------------------------------------|-------------------------------|
| Every post takes 3 attempts    | The critic is not being satisfied, or its verdict is unparseable.                           | critic repeated the SAME note |
| Posts revised into nonsense    | A malformed verdict reached write_node as an instruction.                                   | returned a MALFORMED verdict  |
| A post exceeds its limit       | Cannot happen via the loop — check the consistency rewrite, which does not re-run the gate. | length check FAILED           |
| A platform is missing entirely | One branch did not return.                                                                  | produced no draft for         |
| Posts contradict each other    | The consistency pass failed and fell back to an empty report.                               | comparison FAILED             |

Set LOG\_LEVEL=DEBUG to add per-call detail: draft excerpts, verdict parsing decisions, model tier routing, and state reducer merges. At INFO the log shows one line per box traversal.

Files referenced: src/node/writer\_node.py · src/node/self\_check\_node.py · src/node/accept\_node.py · src/node/consistency\_node.py · src/routes/routing.py · src/agent.py · src/schemas/state.py · src/graphs/graph.py · configs/llms.py